MTM4692

Spring 2026

Fettah KIRAN

Department of AI and Data Engineering

fkiran@yildiz.edu.tr

Cursors

Row-by-Row Processing in MySQL & SQLite Alternatives

📋 Today's Agenda (1:00 PM – 3:45 PM)

Time	Topic	Type
1:00 – 1:05	Recap Quiz	Review
1:05 – 1:20	What are Cursors?	**Theory** 📖
1:20 – 1:45	Cursor Lifecycle	**Theory** 📖
1:45 – 2:10	Cursor with NOT FOUND Handler	**Theory** 📖
2:10 – 2:25	☕ Break (15 min)	
2:25 – 2:50	Practical Cursor Examples	**Practice** 💻
2:50 – 3:05	Cursors vs Set-Based Operations	**Theory** 📖
3:05 – 3:20	SQLite Alternatives	**Practice** 💻
3:20 – 3:40	Practice & Homework	**Practice** 💻
3:40 – 3:45	Recap & next week preview	Wrap-up

Cursors

A **cursor** is a database object that enables **row-by-row processing** in SQL procedures. Think of it as a pointer that moves through a result set one row at a time, allowing you to fetch, process, and act on each row individually. Cursors are essential when you need custom logic per row, but should be avoided when a single SQL statement can do the job faster.

Video: https://www.youtube.com/watch?v=RHRjLd0bEaQ

The Problem: Row-by-Row Processing

SQL is designed for **set-based** operations:

```
-- Operates on ALL rows at once
UPDATE student SET gpa = gpa + 0.1
WHERE dept_id = 1;
```

But sometimes you need to process rows **one at a time**:

- Complex calculations that depend on **previous rows**
- **Custom logic** per row (different action for each)
- Generating **sequential numbers** or reports
- Calling **external procedures** for each row

Cursors provide row-by-row processing!

Cursor = Pointer to a Result Set

Think of a cursor as a **pointer** (like a bookmark) that moves through query results one row at a time:

```
Result Set:                Cursor Position:
```

Alice	3.95
Bob	3.80
Carol	3.20
Dave	2.50

```
← Cursor points here (FETCH)
```

After each `FETCH`, the cursor moves to the **next row**.

Cursor Analogy: Reading a Book

Book Reading	Cursor Processing
Open the book	DECLARE + OPEN cursor
Read one page	FETCH a row
Turn to next page	Next FETCH
Reached last page	NOT FOUND condition
Close the book	CLOSE cursor

When to & NOT to Use Cursors

✅ Appropriate Uses	❌ When NOT to use
Complex row-by-row calculations	Simple updates/deletes on multiple rows
Calling another procedure per row	Aggregations (SUM, AVG, COUNT)
Generating formatted reports	JOINs and filtering
Processing each row differently	Any operation SQL can do in one statement

> **Rule of Thumb**: If you can do it with a single SQL statement, don't use a cursor!

Cursor Lifecycle

The Four Steps

```
1. DECLARE cursor          Define the SELECT query
   ↓
2. OPEN cursor             Execute the query
   ↓
3. FETCH cursor INTO vars  Get one row at a time
   ↓ (loop)
4. CLOSE cursor            Release resources
```

SQL Pseudo Code:

```
-- 1. DECLARE the cursor with a SELECT query
DECLARE cursor_name CURSOR FOR
    SELECT col1, col2 FROM table_name;

-- 2. OPEN the cursor (executes the query)
OPEN cursor_name;

-- 3. FETCH and LOOP through each row
my_loop: LOOP
    FETCH cursor_name INTO var1, var2;
    IF done THEN LEAVE my_loop; END IF;
    -- Process the row here
END LOOP;

-- 4. CLOSE the cursor
CLOSE cursor_name;
```

🧠 **Exam Focus**: Know these four steps in order!

Step 1: DECLARE

```
DECLARE cursor_name CURSOR FOR
    SELECT column1, column2
    FROM table_name
    WHERE condition;
```

- Defines **which query** the cursor will iterate over
- Does **not** execute the query yet
- Must be declared **after** variable declarations
- Must be declared **before** handler declarations

Declaration Order in MySQL

```
BEGIN
    -- 1. Variables FIRST
    DECLARE v_name VARCHAR(50);
    DECLARE v_gpa DECIMAL(3,2);
    DECLARE v_done INT DEFAULT 0;

    -- 2. Cursors SECOND
    DECLARE student_cursor CURSOR FOR
        SELECT first_name, gpa FROM student;

    -- 3. Handlers THIRD
    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET v_done = 1;

    -- 4. Code LAST
    OPEN student_cursor;
    -- ...
END; //This order is **mandatory** in MySQL!
```

Step 2: OPEN

```
OPEN cursor_name;
```

- **Executes** the SELECT query
- Creates the **result set** in memory
- Positions the cursor **before** the first row
- Must be called before FETCH

Step 3: FETCH

```
FETCH cursor_name INTO var1, var2, ...;
```

- Retrieves the **current row** into variables
- Advances the cursor to the **next row**
- Number of variables must match number of SELECT columns
- When no more rows: triggers `NOT FOUND` condition

```
-- Example: fetch two columns into two variables
FETCH student_cursor INTO v_name, v_gpa;
```

Step 4: CLOSE

```
CLOSE cursor_name;
```

- Releases the cursor **resources** (memory)
- The cursor **cannot** be used again (must re-OPEN)
- Always close cursors when done!

🧠 Exam Focus — Lifecycle Summary

Step	SQL	What Happens
1	`DECLARE ... CURSOR FOR`	Define the query
2	`OPEN cursor`	Execute query, create result set
3	`FETCH cursor INTO`	Get next row, advance pointer
4	`CLOSE cursor`	Free resources

> Exam Tip: "Which step executes the query?" → **OPEN**
> "Which step retrieves a row?" → **FETCH**

NOT FOUND Handler

The NOT FOUND Condition

When FETCH reaches the **end** of the result set, MySQL raises `NOT FOUND`.

Without a handler → error!

With a handler → graceful termination:

```
DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET v_done = 1;
```

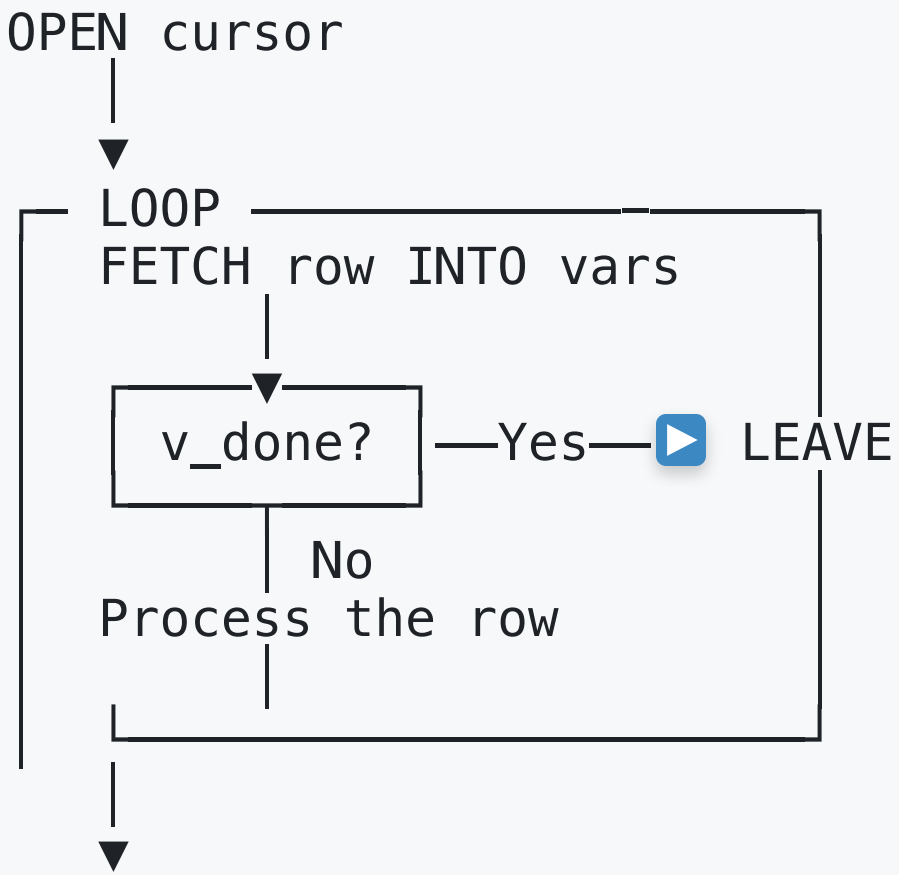
- `CONTINUE` means: handle the condition and **continue** execution
- `NOT FOUND` is the condition (no more rows)
- `SET v_done = 1` is the action (set a flag)

The Standard Cursor Loop Pattern

```
OPEN cur;
read_loop: LOOP
    FETCH cur INTO v_name, v_gpa;
    IF v_done = 1 THEN LEAVE read_loop; END IF;
    -- Process row here
END LOOP;
CLOSE cur;
```

See full example in **CODING PRACTICE** section.

Loop Pattern Explained



Alternative: WHILE Loop Pattern

```
OPEN cur;
FETCH cur INTO v_name, v_gpa;
WHILE v_done = 0 DO
    -- Process row
    FETCH cur INTO v_name, v_gpa;
END WHILE;
CLOSE cur;
```

LOOP + LEAVE pattern is more common in MySQL.

🧠 Exam Focus — NOT FOUND Handler

> **Q:** What happens if you FETCH past the last row without a handler?
> **A:** MySQL raises an error and the procedure fails!

> **Q:** What does `DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;` do?
> **A:** When FETCH finds no more rows, it sets `done = 1` and continues execution (allowing the LEAVE check to exit the loop).

Practical Cursor Examples

Example 1: Grade Report Generator

```
DECLARE cur CURSOR FOR
    SELECT first_name, gpa FROM student ORDER BY gpa DESC;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

OPEN cur;
LOOP
    FETCH cur INTO v_name, v_gpa;
    IF v_done THEN LEAVE; END IF;
    SET v_grade = CASE WHEN v_gpa >= 3.7 THEN 'A' ...
    INSERT INTO grade_report VALUES (...);
END LOOP;
```

See full example in CODING PRACTICE section.

Example 2: Conditional Updates per Row

```
DECLARE cur CURSOR FOR SELECT student_id, gpa, dept_id FROM student;

OPEN cur;
update_loop: LOOP
    FETCH cur INTO v_id, v_gpa, v_dept;
    IF v_done THEN LEAVE update_loop; END IF;

    IF v_dept = 1 THEN
        UPDATE student SET gpa = LEAST(gpa + 0.2, 4.0) ...
END LOOP;
```

See full example in CODING PRACTICE section.

Example 3: Running Total with Cursor

```
DECLARE cur CURSOR FOR
    SELECT order_id, total_amount FROM orders ORDER BY order_date;

OPEN cur;
calc_loop: LOOP
    FETCH cur INTO v_id, v_amount;
    IF v_done THEN LEAVE calc_loop; END IF;
    SET v_running = v_running + v_amount;
    INSERT INTO running_totals VALUES (...);
END LOOP;
```

See full example in CODING PRACTICE section.

Example 4: Cursor with Error Handling

```
DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET v_err = 1;

OPEN cur;
enroll_loop: LOOP
    FETCH cur INTO v_sid;
    IF v_done THEN LEAVE enroll_loop; END IF;
    INSERT INTO enrollment ... ;
    IF v_err = 0 THEN SET p_success = p_success + 1; END IF;
END LOOP;
```

See full example in CODING PRACTICE section.

Example 5: Nested Cursor

```
OPEN dept_cur;
dept_loop: LOOP
    FETCH dept_cur INTO v_dept_id, v_dept_name;
    IF v_done_dept THEN LEAVE dept_loop; END IF;

    BEGIN
        DECLARE stu_cur CURSOR FOR
            SELECT first_name, gpa FROM student
            WHERE dept_id = v_dept_id;
        SET v_done_stu = 0;
        OPEN stu_cur;
        -- Inner loop
    END;
END LOOP;
```

See full example in CODING PRACTICE section.

🧠 Exam Focus — Cursor Patterns

> **Inner cursor** must be declared in a nested `BEGIN...END` block
>
> Reset the `done` flag before opening the inner cursor!
>
> Always CLOSE cursors to prevent resource leaks

Cursors vs Set-Based Operations

Performance Comparison

Cursor approach (SLOW):

```
OPEN cur;
LOOP
    FETCH cur INTO v_id, v_gpa;
    UPDATE student SET gpa = gpa + 0.1 WHERE student_id = v_id;
END LOOP;
```

Set-based approach (FAST):

```
UPDATE student SET gpa = gpa + 0.1;
```

Set-based is typically 10-100x faster!

When Set-Based Wins

Task	Cursor	Set-Based Winner
Update all rows	LOOP + UPDATE each	`UPDATE ... SET ...`
Calculate totals	LOOP + accumulate	`SUM()`, `AVG()`
Find max/min	LOOP + compare	`MAX()`, `MIN()`
Filter rows	LOOP + IF check	`WHERE` clause
Running total	LOOP + accumulate	Window function `SUM() OVER()`

Running Total: Cursor vs Window Function

Cursor: ~20 lines of code

```
OPEN cur;
LOOP
    FETCH cur INTO v_id, v_amount;
    SET v_running = v_running + v_amount;  -- Accumulate
    INSERT INTO result VALUES (...);
END LOOP;
```

Window Function: 1 line, set-based!

```
SELECT order_id, total_amount,
       SUM(total_amount) OVER (ORDER BY order_date) AS running_total
FROM orders;
```

Window functions eliminate most cursor needs!

When Cursors are Necessary

1. **Calling another procedure** for each row

```
FETCH cur INTO v_id;
CALL process_individual(v_id, @result);
```

2. **Different actions** based on complex conditions

```
IF v_status = 'A' THEN CALL action_a(v_id);
ELSEIF v_status = 'B' THEN CALL action_b(v_id);
END IF;
```

3. **External system calls** per row

4. **Sequential numbering** with gaps

🧠 Exam Focus — Cursors vs Set-Based

> **Always prefer set-based** operations when possible
>
> Cursors are slower because they process **one row at a time**
>
> **Window functions** have replaced many traditional cursor uses
>
> Use cursors only when you **must** process each row differently

SQLite Alternatives to Cursors

SQLite Has No Cursor Support

SQLite does **not** support:

- ❌ DECLARE CURSOR
- ❌ OPEN / FETCH / CLOSE
- ❌ NOT FOUND handler
- ❌ Stored procedures (where cursors live)

SQLite alternatives:

- ✅ Window functions (running totals, ranks)
- ✅ Recursive CTEs (sequential processing)
- ✅ Application-level loops (Python, etc.)
- ✅ Correlated subqueries

Alternative 1: Window Functions

```sql
-- Instead of cursor for running totals:
SELECT order_id, total_amount,
       SUM(total_amount) OVER (ORDER BY order_date) AS running_total
FROM orders;
```

See full examples in CODING PRACTICE section.

Alternative 2: Recursive CTE as Cursor

```
WITH RECURSIVE ranked AS (
    SELECT student_id, first_name, ROW_NUMBER() OVER (...) AS rn
    FROM student
),
process AS (
    SELECT * FROM ranked WHERE rn = 1
    UNION ALL
    SELECT r.* FROM ranked r JOIN process p ON r.rn = p.rn + 1
)
SELECT * FROM process;
```

See full examples in CODING PRACTICE section.

Alternative 3: Python Application Loop

```
cursor.execute('SELECT student_id, gpa FROM student')
for sid, gpa in cursor:
    if gpa >= 3.5:
        # Process with custom logic
        print(f"Honor: {sid}")
```

See full examples in CODING PRACTICE section.

Alternative 4: GROUP_CONCAT for Reporting

```sql
SELECT dept_name, COUNT(*) AS students,
       GROUP_CONCAT(first_name, ', ') AS student_list,
       ROUND(AVG(gpa), 2) AS avg_gpa
FROM department d
LEFT JOIN student s ON d.dept_id = s.dept_id
GROUP BY d.dept_id;
```

See full examples in CODING PRACTICE section.

Comparison: MySQL Cursors vs SQLite

MySQL Cursor Feature	SQLite Alternative
DECLARE CURSOR FOR	Python `cursor.execute()`
OPEN	Query execution
FETCH INTO	`for row in cursor`
NOT FOUND handler	End of iterator
CLOSE	`cursor.close()`
Running total loop	`SUM() OVER()`
Row ranking loop	`ROW_NUMBER() OVER()`
Conditional processing	CASE expression

 BREAK (15 minutes)

💻 PART 7 — CODING PRACTICE

Cursor & Set-Based Alternatives

💻 Cursor Loop Pattern (MySQL)

```
-- Basic cursor loop: process each student
DELIMITER //
CREATE PROCEDURE process_students()
BEGIN
    DECLARE v_name VARCHAR(100);
    DECLARE v_gpa DECIMAL(3,2);
    DECLARE v_done INT DEFAULT 0;

    DECLARE cur CURSOR FOR
        SELECT CONCAT(first_name, ' ', last_name), gpa
        FROM student ORDER BY gpa DESC;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO v_name, v_gpa;
        IF v_done = 1 THEN LEAVE read_loop; END IF;

        -- Process each row here
        SELECT v_name, v_gpa;
    END LOOP;
    CLOSE cur;
END //
DELIMITER ;

-- Call it:
CALL process_students();
```

💻 Cursor with Conditional Logic

```
-- Cursor processing with different actions per department
DELIMITER //
CREATE PROCEDURE apply_custom_curve()
BEGIN
    DECLARE v_id INT;
    DECLARE v_gpa DECIMAL(3,2);
    DECLARE v_dept INT;
    DECLARE v_done INT DEFAULT 0;

    DECLARE cur CURSOR FOR
        SELECT student_id, gpa, dept_id FROM student;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

    START TRANSACTION;
    OPEN cur;

    update_loop: LOOP
        FETCH cur INTO v_id, v_gpa, v_dept;
        IF v_done THEN LEAVE update_loop; END IF;

        -- Different curve per department
        IF v_dept = 1 THEN
            UPDATE student SET gpa = LEAST(gpa + 0.2, 4.0)
            WHERE student_id = v_id;
        ELSEIF v_dept = 2 THEN
            UPDATE student SET gpa = LEAST(gpa + 0.1, 4.0)
            WHERE student_id = v_id;
        END IF;
    END LOOP;

    CLOSE cur;
    COMMIT;
END //
DELIMITER ;

CALL apply_custom_curve();
```

💻 Window Functions Replace Cursors (SQLite/MySQL)

```sql
-- INSTEAD OF a cursor that accumulates running totals:
-- Use SUM() OVER() — 100x faster!

SELECT order_id, total_amount,
       SUM(total_amount) OVER (
           ORDER BY order_date
           ROWS UNBOUNDED PRECEDING
       ) AS running_total
FROM orders
ORDER BY order_date;

-- INSTEAD OF a cursor that ranks students per department:
-- Use ROW_NUMBER() OVER()

SELECT first_name, dept_id, gpa,
       ROW_NUMBER() OVER (
           PARTITION BY dept_id ORDER BY gpa DESC
       ) AS dept_rank
FROM student;
```

💻 Conditional Aggregation (SQLite/MySQL)

```sql
-- INSTEAD OF a cursor that calculates scholarship per student:
-- Use CASE + SUM() — set-based!

SELECT d.dept_id, d.dept_name,
       COUNT(*) AS student_count,
       SUM(CASE
           WHEN gpa >= 3.7 THEN 500
           WHEN gpa >= 3.0 THEN 300
           WHEN gpa >= 2.0 THEN 100
           ELSE 0
       END) AS total_scholarship,
       ROUND(AVG(gpa), 2) AS avg_gpa
FROM department d
LEFT JOIN student s ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY d.dept_id;
```

💻 Update: Cursor vs Set-Based

```
-- ❌ SLOW: Cursor approach (row-by-row)
OPEN cur;
LOOP
    FETCH cur INTO v_id, v_gpa;
    IF v_gpa < 2.0 THEN
        UPDATE student SET status = 'probation'
        WHERE student_id = v_id;
    ELSE
        UPDATE student SET status = 'active'
        WHERE student_id = v_id;
    END IF;
END LOOP;

-- ✅ FAST: Set-based approach (one statement)
UPDATE student SET status =
    CASE
        WHEN gpa < 2.0 THEN 'probation'
        ELSE 'active'
    END;
```

💻 Declaration Order (Common Error)

```
-- ❌ WRONG ORDER:
BEGIN
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;  -- Too early!
    DECLARE done INT DEFAULT 0;                            -- After handler!
    DECLARE cur CURSOR FOR SELECT ...;                     -- After handler!
END;

-- ✅ CORRECT ORDER:
BEGIN
    -- 1. Variables FIRST
    DECLARE done INT DEFAULT 0;
    DECLARE v_name VARCHAR(50);

    -- 2. Cursors SECOND
    DECLARE cur CURSOR FOR SELECT first_name FROM student;

    -- 3. Handlers THIRD
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    -- 4. Executable code LAST
    OPEN cur;
    ...
END;
```

Homework Practices

Choose your database platform and complete the exercises below:

MySQL Students (Use Cursor Approach):

1. **Basic Cursor Loop**
   - Write a procedure that uses a cursor to iterate students
   - Display student names and GPAs one row at a time
   - Use the cursor lifecycle: DECLARE → OPEN → FETCH → CLOSE
   - *See: PRACTICE EXERCISE 1 in week12_practice.sql*

2. **Cursor with CASE Logic**
   - Use a cursor to process each student
   - Assign letter grades based on GPA using CASE statements
   - Process row-by-row with conditional IF logic
   - *See: PRACTICE EXERCISE 2 in week12_practice.sql*
3. **Cursor with Accumulation**
   - Write a procedure with a cursor that counts and numbers students
   - Use an accumulator variable to track progress through the loop
   - Generate a sequential numbered list
   - *See: PRACTICE EXERCISE 3 in week12_practice.sql*

SQLite Students (Use Set-Based Approach):

1. **Window Functions for Running Totals**
   - Use `SUM() OVER (ORDER BY ...)` to create running totals
   - Compare the result with what a cursor loop would produce
   - Observe the dramatic speed improvement!
   - *See: SECTION 1 in week12_practice.sql*
2. **Conditional Aggregation (Scholarship Calculation)**
   - Use `CASE` within `SUM()` to calculate totals by performance tier
   - Calculate department-level scholarship distributions by GPA brackets
   - Much faster than row-by-row cursor processing!
   - *See: SECTION 3 in week12_practice.sql*

🧠 Key Exam Concepts

Topic	What to Remember
Lifecycle	DECLARE → OPEN → FETCH → CLOSE
Declaration Order	Variables → Cursor → Handler
NOT FOUND	Triggered when FETCH reaches end
CONTINUE HANDLER	Handles condition, continues execution
LEAVE label	Exits the named loop
OPEN	Executes the query
FETCH	Gets one row, advances pointer
Set-based > Cursor	Always prefer single SQL statements

Next Week Preview

Week 13: ACID Transactions & Triggers

- ACID properties (Atomicity, Consistency, Isolation, Durability)
- Transaction control: COMMIT, ROLLBACK, SAVEPOINT
- Isolation levels
- BEFORE and AFTER triggers
- NEW and OLD row references
- Trigger use cases

Thank You!

Questions?